

Documento de Especificaciones Técnicas

Agenda Dinámica — Matriz de Eisenhower

Proyecto: Sistema Web de Gestión del Tiempo y Priorización

Líder Técnico / Desarrollador: Daniel Alberto Morán Vílchez

Versión: 1.0

Fecha: Julio 2026

1. Visión y Alcance del Proyecto

El objetivo principal es desarrollar una aplicación web personal orientada a la gestión del tiempo y aumento de la productividad. El sistema se basará en la **Matriz de Eisenhower**, permitiendo clasificar tareas dinámicamente según su urgencia e importancia.

El alcance de la primera iteración (MVP) abarca la construcción de la interfaz de usuario para la manipulación de las tareas en los cuatro cuadrantes, establecimiento de autenticación segura, almacenamiento de datos en la nube en tiempo real y la sincronización mediante API con Google Calendar para la automatización de notificaciones nativas.

2. Arquitectura y Stack Tecnológico

La arquitectura del sistema ha sido diseñada priorizando la reactividad fluida en el frontend y utilizando servicios gestionados (BaaS) en su capa gratuita para el backend, garantizando un despliegue rápido, eficiente y escalable.

- **Frontend:** Vue 3 (Composition API). Seleccionado por su capacidad excepcional para manejar el estado reactivo del DOM. Ideal para mover tareas entre cuadrantes de forma instantánea sin manipulaciones complejas.
- **Base de Datos:** Cloud Firestore (Firebase). Base de datos NoSQL documental, elegida para aprovechar las suscripciones en tiempo real (onSnapshot), sincronizando los dispositivos del usuario sin peticiones manuales.
- **Autenticación:** Firebase Auth para el acceso principal a la plataforma (Correo electrónico / Contraseña) combinado con OAuth 2.0 de Google para obtener exclusivamente los permisos de escritura en la agenda del usuario.
- **Hosting:** Firebase Hosting, integrado directamente con el entorno de construcción y empaquetado de Vue (Vite).
- **Integración Externa:** Google Calendar API (v3) mediante peticiones REST seguras.

3. Modelo de Datos (Estructura Base)

El modelo adapta la lógica de negocio de agendas gerenciales dinámicas hacia una estructura orientada a documentos NoSQL. La colección principal tareas en Firestore tendrá la siguiente composición:

```
{
  "id": "string (uuid generado auto.)",
  "titulo": "string",
  "descripcion": "string",
  "es_urgente": "boolean",
  "es_importante": "boolean",
  "fecha_inicio": "timestamp",
  "fecha_vencimiento": "timestamp",
  "estado": "string (Pendiente | En curso | Finalizado)",
  "area_contexto": "string",
  "calendar_event_id": "string (opcional, devuelto por Google)"
}
```

Nota técnica: La posición visual en la interfaz (el cuadrante I, II, III o IV) no se guarda como una cadena de texto, sino que se computa dinámicamente en el frontend de Vue basándose en la combinación de los valores booleanos es_urgente y es_importante.

4. Requerimientos Funcionales (RF)

ID	Requerimiento	Descripción
RF-01	Autenticación Híbrida	El sistema permitirá el login inicial con Firebase Auth. Posteriormente, la interfaz ofrecerá la opción de "Vincular Google Calendar" ejecutando el flujo OAuth 2.0 y almacenando los tokens de acceso/refresco.
RF-02	Gestión de Tareas (CRUD)	El usuario podrá crear, leer, actualizar y eliminar tareas. La modificación de los atributos de urgencia e importancia disparará la reubicación visual de la tarea en tiempo real.
RF-03		

	Vista de Matriz de Eisenhower	Renderizado principal dividido en 4 cuadrantes lógicos. Los cuadrantes superiores (I y II) integrarán opciones rápidas de calendarización.
RF-04	Sincronización Bidireccional Base	Al establecer una ventana de tiempo para una tarea, el sistema enviará un payload a Google Calendar. El eventId resultante se guardará en Firestore para futuras actualizaciones.
RF-05	Notificaciones Nativas	El payload de creación del evento incluirá obligatoriamente el array reminders.overrides para asegurar que Google procese y dispare alertas (push/email) al usuario en sus dispositivos móviles y de escritorio.

5. Requerimientos No Funcionales (RNF)

- **RNF-01: Reactividad SPA.** La aplicación funcionará como Single Page Application. Ninguna acción de guardado o filtrado deberá requerir la recarga del navegador.
- **RNF-02: Disponibilidad y Rendimiento.** Al utilizar Firebase, se hereda la alta disponibilidad de la infraestructura de Google Cloud.
- **RNF-03: Seguridad de Datos.** Se implementarán *Firebase Security Rules* restrictivas para asegurar que un usuario únicamente tenga permisos de lectura y escritura sobre los documentos cuyo user_id coincida con su token de autenticación.
- **RNF-04: Diseño Adaptativo (Responsive).** La grilla CSS de la matriz 2x2 debe colapsar fluidamente a 1 columna en resoluciones móviles, garantizando una excelente usabilidad en cualquier formato de pantalla.

6. Estrategia de Implementación y Fases

1. **Fase 1: Pruebas de Concepto (PoC) Backend.** Construir un script aislado para autorizar OAuth 2.0 y lograr inyectar exitosamente un evento de prueba con notificaciones en Calendar.
2. **Fase 2: Arquitectura Base.** Inicialización del proyecto Vue 3, configuración de la consola de Firebase, esquemas de Firestore y conexión inicial (SDK).
3. **Fase 3: Desarrollo de UI/UX.** Maquetación de la matriz, tarjetas de tareas, formularios interactivos y modales de configuración.

4. **Fase 4: Integración del Flujo Completo.** Enlazar los disparadores reactivos del frontend con las operaciones de escritura en Firestore y las llamadas a la API de Google.
5. **Fase 5: Testing y Despliegue.** Revisión de reglas de seguridad, compilación de producción y publicación final en Firebase Hosting.